



Sprint 12 · Lenguaje, Secuencias y Modelos Generativos

S12.1 · RNN, LSTM, GRU y Series Temporales

Un recorrido por las arquitecturas recurrentes y las técnicas fundamentales para modelar datos que evolucionan en el tiempo. Desde las redes RNN clásicas hasta LSTM y GRU, pasando por la validación temporal y la comparación con métodos estadísticos.

Contenidos del Sprint

Este sprint aborda los bloques esenciales para trabajar con secuencias y series temporales en aprendizaje profundo.

01

Datos secuenciales

Naturaleza y estructura de las secuencias

03

El problema del gradiente

Vanishing y exploding gradient

05

Ventanas y series temporales

Tendencia, estacionalidad y validación

02

Redes recurrentes (RNN)

Arquitectura y funcionamiento básico

04

LSTM y GRU

Soluciones avanzadas con puertas

06

Comparación con modelos clásicos

ARIMA, Prophet vs. redes neuronales



1 · Datos Secuenciales

No todos los datos son independientes entre sí. En muchos problemas reales, **el orden importa**: lo que ocurrió ayer influye en lo que ocurre hoy. Este tipo de información se denomina **dato secuencial**.

¿Qué hace especial a una secuencia?

A diferencia de los datos tabulares clásicos, las secuencias tienen **dependencias temporales o posicionales** que los modelos deben capturar.



Series temporales

Temperatura diaria, precio de acciones, consumo eléctrico. El instante t depende de los instantes previos.



Audio y vídeo

Los fotogramas o las muestras de audio forman secuencias donde la coherencia temporal es esencial.



Lenguaje natural

En una frase, cada palabra depende de las anteriores. El contexto es fundamental para interpretar el significado.



Secuencias biológicas

Cadenas de ADN o proteínas donde la posición de cada elemento determina la función del conjunto.

2 · Redes Recurrentes (RNN)

Una **red neuronal recurrente** (RNN) procesa las entradas una a una, manteniendo un **estado oculto** h_t que actúa como memoria a corto plazo. En cada paso de tiempo, la red combina la entrada actual x_t con el estado anterior h_{t-1} para producir el nuevo estado:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Este mecanismo permite, en teoría, que la red recuerde información de pasos anteriores. Sin embargo, en la práctica esta memoria se **desvanece rápidamente** con secuencias largas.

Idea clave

La RNN comparte los mismos pesos W en cada paso de tiempo. Esto la hace eficiente en parámetros, pero genera el problema del gradiente que veremos a continuación.

- Entrada: x_t en cada instante
- Estado oculto: h_t como memoria
- Salida: y_t opcional en cada paso

Arquitecturas RNN más comunes

Según el número de entradas y salidas, una RNN puede adoptar distintas configuraciones para diferentes tareas.

One-to-Many

Una entrada genera una secuencia de salidas.
Ejemplo: **generación de texto** a partir de una semilla.

Many-to-One

Una secuencia produce una única salida.
Ejemplo: **análisis de sentimiento** de una frase completa.

Many-to-Many

Secuencia a secuencia. Ejemplo: **traducción automática** o etiquetado morfosintáctico.

Bidireccional

Dos RNN, una en cada dirección. Captura contexto pasado **y** futuro simultáneamente.



3 · El Problema del Gradiente en Secuencias

El talón de Aquiles de las RNN clásicas: los gradientes se desvanecen o explotan al retropropagarse a través del tiempo, impidiendo aprender dependencias a largo plazo.

Vanishing y Exploding Gradient

Vanishing Gradient

Durante la retropropagación a través del tiempo (**BPTT**), el gradiente se multiplica repetidamente por la matriz de pesos W_h . Si sus valores son menores que 1, el gradiente **se aproxima a cero** exponencialmente conforme nos alejamos en el tiempo. La red "olvida" eventos pasados y no puede aprender dependencias largas.

Exploding Gradient

Si los pesos son mayores que 1, el gradiente **crece exponencialmente**, desestabilizando el entrenamiento con actualizaciones gigantescas. Se mitiga con **gradient clipping**: si la norma del gradiente supera un umbral, se reescala para limitar su magnitud.



El vanishing gradient es el motivo principal por el que las RNN simples fracasan con secuencias de más de 10–20 pasos. LSTM y GRU nacen precisamente para resolver este problema.

4 · LSTM: Long Short-Term Memory

Propuesta por Hochreiter y Schmidhuber en 1997, la LSTM introduce un mecanismo de **puertas** que regula qué información conservar, actualizar u olvidar, resolviendo el problema del gradiente de forma elegante.



Las tres puertas de la LSTM

El núcleo de la LSTM es su **célula de estado** C_t , protegida y modulada por tres puertas aprendibles.



Puerta de olvido (*forget gate*)

Decide qué parte del estado anterior C_{t-1} se descarta. Genera valores entre 0 (olvidar todo) y 1 (recordar todo) mediante una sigmoide aplicada a $[h_{t-1}, x_t]$.



Puerta de entrada (*input gate*)

Controla qué nueva información se incorpora al estado. Combina una sigmoide (¿cuánto?) con una tangente hiperbólica (¿qué valor nuevo?) para actualizar C_t .



Puerta de salida (*output gate*)

Determina qué parte del estado de la célula se expone como estado oculto h_t . Filtra la información relevante para la predicción actual.

Gracias a las conexiones aditivas en C_t , el gradiente puede fluir sin degradarse durante muchos pasos de tiempo.

GRU: Gated Recurrent Unit

Propuesta por Cho et al. en 2014, la **GRU** simplifica la LSTM fusionando las puertas de olvido y entrada en una única **puerta de actualización** (*update gate*), y eliminando la célula de estado separada.

Con **menos parámetros**, la GRU es más rápida de entrenar y generaliza bien en conjuntos de datos de tamaño medio. En muchos benchmarks de series temporales su rendimiento es comparable al de la LSTM.

Puertas de la GRU

Update gate z_t

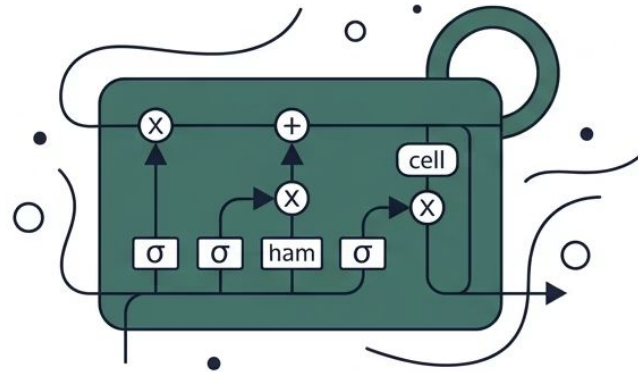
Decide cuánta información del pasado conservar frente a la nueva entrada. Reemplaza a las puertas de olvido y entrada de la LSTM.

Reset gate r_t

Controla cuánto del estado oculto pasado se usa para calcular el nuevo estado candidato. Permite olvidar contextos irrelevantes.

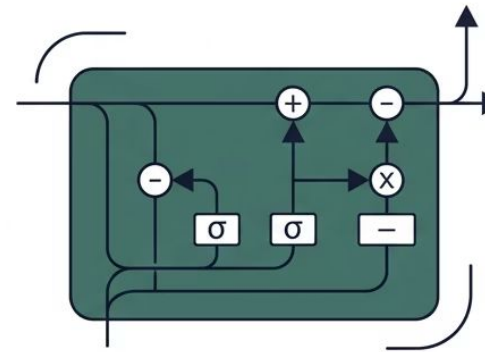
LSTM vs. GRU: ¿Cuál elegir?

LSTM



COMPLEJIDAD: ALTA (4 puertas, célula de estado);
ENTRENAMIENTO: MÁS LENTA;
SECUENCIAS LARGAS: EXCELENTE;
DATOS: MEJOR CON GRANDES DATASETS

GRU



COMPLEJIDAD: BAJA (2 puertas, sin célula separada);
ENTRENAMIENTO: MÁS RÁPIDA;
SECUENCIAS LARGAS: BUENA (ligeramente inferior);
DATOS: MEJOR CON DATSETS MEDIANOS/PEQUEÑOS

En la práctica, se recomienda probar ambas arquitecturas. La GRU es un buen punto de partida por su eficiencia; la LSTM puede aportar ventajas cuando la secuencia tiene dependencias muy largas o el conjunto de datos es grande.



5 · Ventanas Deslizantes

La técnica de **ventana deslizante** (*sliding window*) es el puente entre una serie temporal raw y el formato entrada–salida que necesita una red neuronal.

Cómo funciona la ventana deslizante

Dada una serie temporal $X = [x_1, x_2, \dots, x_T]$, se define una ventana de tamaño w . Se desplaza por la serie generando pares:

- **Entrada:** $[x_t, x_{t+1}, \dots, x_{t+w-1}]$ — los últimos w valores conocidos
- **Objetivo:** x_{t+w} — el valor a predecir (o varios, en predicción multistep)

El tamaño de la ventana es un **hiperparámetro crítico**: muy pequeño pierde contexto, muy grande añade ruido y coste computacional. Se ajusta mediante validación cruzada temporal.

Consideraciones prácticas

- Normalizar dentro de cada ventana (MinMax o Z-score) para estabilizar el entrenamiento
- Con datos diarios, ventanas de 7, 14 o 30 días suelen capturar patrones semanales o mensuales
- En predicción multistep, distinguir entre estrategia directa, recursiva y MIMO



6 · Series Temporales: Tendencia y Estacionalidad

Antes de modelar, conviene descomponer la serie en sus componentes estructurales. Entenderlos mejora la elección del modelo y la ingeniería de características.

Componentes de una Serie Temporal



Tendencia

Dirección general a largo plazo: creciente, decreciente o estacionaria. Puede ser lineal o no lineal. Se extrae mediante medias móviles o regresión.



Estacionalidad

Patrones periódicos y predecibles: ventas más altas en Navidad, mayor consumo eléctrico en verano. El período puede ser diario, semanal o anual.



Residuo (ruido)

Lo que queda tras eliminar tendencia y estacionalidad. Idealmente es ruido blanco, sin estructura. Si tiene patrón, el modelo aún no captura toda la información.



Ciclo

Fluctuaciones no periódicas de larga duración, como ciclos económicos. A diferencia de la estacionalidad, su período no es fijo ni constante.

La descomposición clásica puede ser **aditiva** ($Y = T + S + R$) o **multiplicativa** ($Y = T \times S \times R$), según si los componentes se suman o se escalan entre sí.

7 · Validación Temporal

En series temporales **no se puede usar validación cruzada aleatoria**. Mezclar pasado y futuro introduce *data leakage*: el modelo "vería" el futuro durante el entrenamiento, inflando artificialmente las métricas.

La solución es respetar siempre el **orden cronológico**: entrenar en el pasado y evaluar en el futuro.

Estrategias correctas

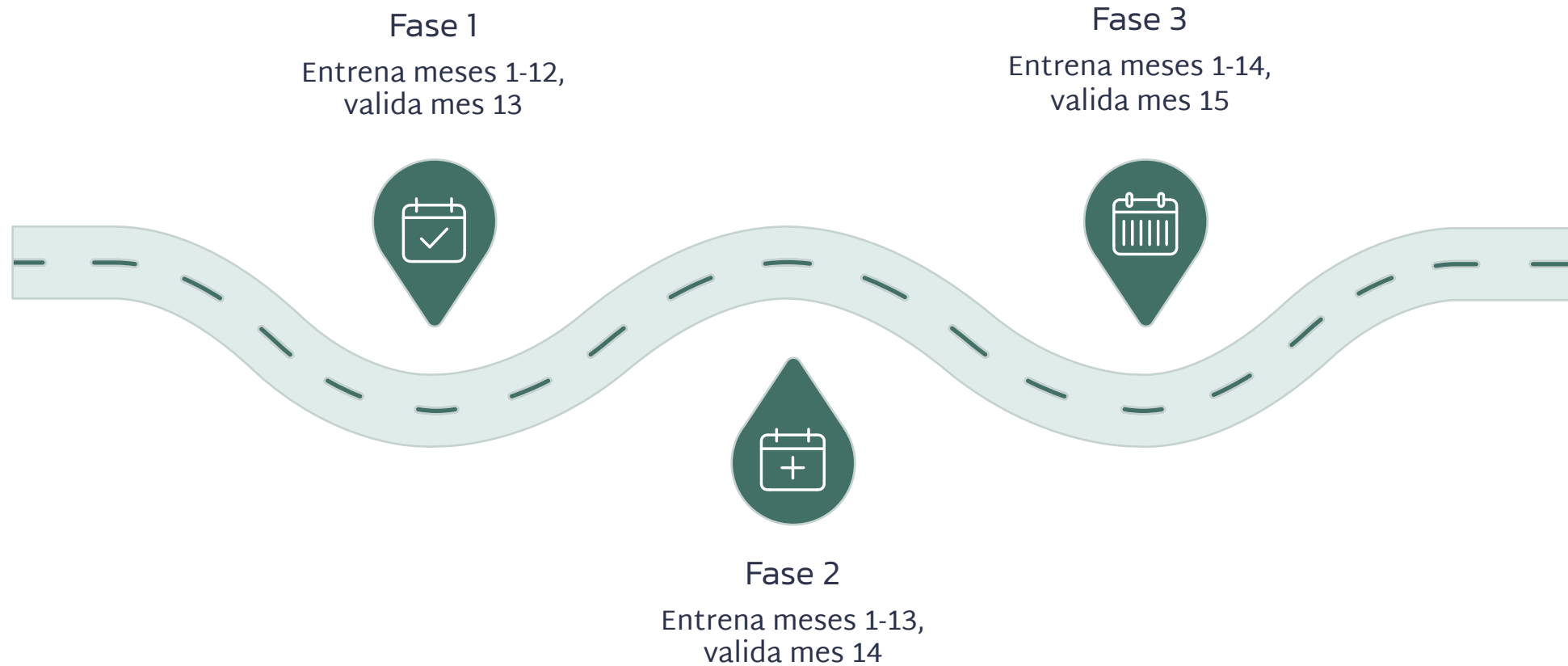
→ Hold-out temporal

División fija: los últimos k meses como test.
Simple pero sensible al período elegido.

→ Walk-forward validation

Ventana de entrenamiento que crece progresivamente. Más robusta y representativa del uso real del modelo en producción.

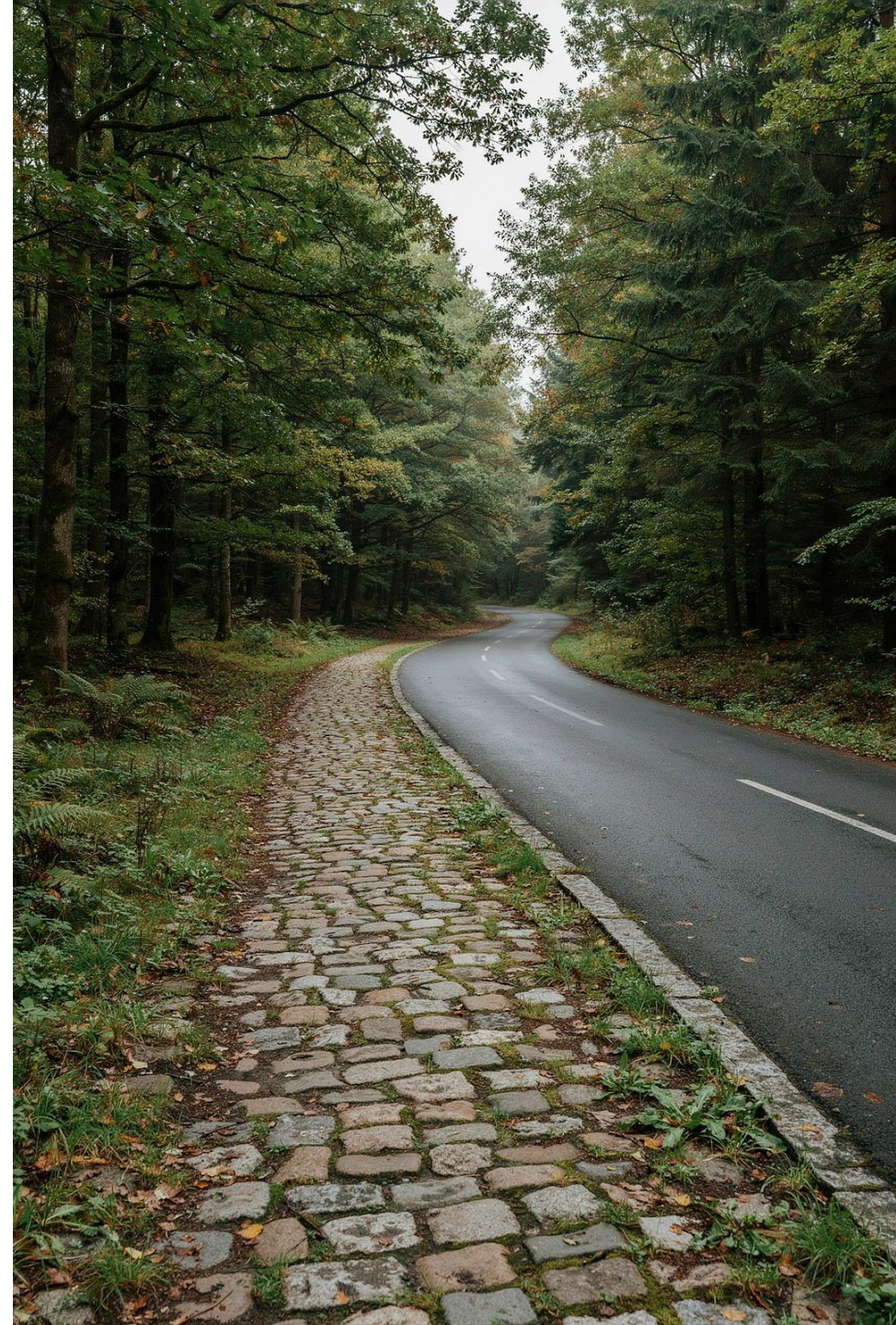
⚠ Nunca uses `train_test_split` con `shuffle=True` en series temporales. El orden cronológico es sagrado.



En cada iteración del **walk-forward**, la ventana de entrenamiento se amplía con el período de validación anterior. Esto simula fielmente cómo se reentrenan los modelos en producción conforme llegan nuevos datos.

8 · Comparación con Modelos Clásicos

Las redes recurrentes no son siempre la mejor opción. Los modelos estadísticos clásicos siguen siendo altamente competitivos en muchos escenarios de series temporales reales.



Modelos Clásicos vs. Redes Recurrentes

Criterio	ARIMA/SARIMA	Prophet	LSTM / GRU
Datos necesarios	Pocos (<500 obs.)	Pocos-medios	Muchos (>1000 obs.)
Interpretabilidad	Alta	Alta	Baja (caja negra)
Estacionalidad	SARIMA la modela explícitamente	Excelente soporte múltiple	Aprendida implícitamente
Relaciones no lineales	Limitada	Limitada	Muy alta
Series multivariadas	VAR (extensión)	No directamente	Nativa
Coste de entrenamiento	Muy bajo	Bajo	Alto (GPU recomendada)

📌 Regla práctica: empieza siempre con un modelo clásico como baseline. Si las redes recurrentes no lo superan significativamente, el modelo simple gana por navaja de Occam.

Resumen y Takeaways del Sprint

Las ideas clave que debes llevarte de esta sesión:

- 1** Las secuencias requieren arquitecturas especiales
Las redes densas ignoran el orden; las RNN lo explotan mediante el estado oculto compartido entre pasos de tiempo.
- 2** LSTM y GRU resuelven el vanishing gradient
Sus puertas aprendibles permiten retener o descartar información selectivamente, habilitando dependencias largas.
- 3** La ventana deslizante transforma series en datasets supervisados
Elegir bien su tamaño es tan importante como elegir la arquitectura del modelo.
- 4** La validación debe respetar el tiempo
Walk-forward validation es el estándar profesional para evaluar modelos de series temporales sin data leakage.
- 5** Los modelos clásicos siguen siendo referencia
ARIMA y Prophet son baselines sólidos. Las redes recurrentes brillan con datos abundantes y relaciones no lineales complejas.